Keygraph

CONFIDENTIAL

Security Assessment Report

TARGET
http://host.docker.internal:4173

DATE
2026-10-14

STATUS
Complete

TESTER
Shannon
AI Pentester by Keygraph

This document contains sensitive security findings. Handle in accordance with your organization's data classification policy.

Contents

Executive Summary . . . 3
Scope . . . 3
Identified Vulnerabilities by Type . . . 3
XSS . . . 3
Authentication . . . 3
Authorization . . . 3
Summary . . . 4
Findings Overview . . . 5
XSS Findings (1 finding) . . . 6
XSS-01: Reflected XSS in the /search q Parameter . . . 6
Authentication Findings (1 finding) . . . 6
AUTH-01: Missing Authentication on /api/notes/{id} . . . 6
Authorization Findings (1 finding) . . . 7
AUTHZ-01: Missing Ownership Authorization on /api/notes/{id} . . . 7

Executive Summary

Target	http://host.docker.internal:4173
Date	2026-10-14
Tester	Shannon
Assessment Status	Complete

The analysis-only assessment of http://host.docker.internal:4173 on 2026-10-14 identified three high-confidence findings: two assessed as high severity and one assessed as medium severity. The most serious weaknesses are missing authentication and missing object-level ownership authorization on the notes API, while unescaped search input creates reflected XSS; together these issues represent an assessed high overall risk despite the demo service's limited, fictional data set. No finding was confirmed by exploitation.

Scope

Injection, Cross-Site Scripting (XSS), Authentication, Authorization, Server-Side Request Forgery (SSRF)

Identified Vulnerabilities by Type

XSS

- **XSS-01** — Reflected XSS in the /search q Parameter

Authentication

- **AUTH-01** — Missing Authentication on /api/notes/{id}

Authorization

- **AUTHZ-01** — Missing Ownership Authorization on /api/notes/{id}

Summary

0	2	1	0
CRITICAL	HIGH	MEDIUM	LOW

3	0	0
HIGH CONFIDENCE	MEDIUM CONFIDENCE	LOW CONFIDENCE

Total identified **3**

- **1** XSS
- **1** Authentication
- **1** Authorization

Findings Overview

ID	Title	Category	Severity	Confidence
XSS-01	Reflected XSS in the /search q Parameter	XSS	MEDIUM	HIGH CONFIDENCE
AUTH-01	Missing Authentication on /api/notes/{id}	Authentication	HIGH	HIGH CONFIDENCE
AUTHZ-01	Missing Ownership Authorization on /api/notes/{id}	Authorization	HIGH	HIGH CONFIDENCE

XSS Findings (1 finding)

XSS-01: Reflected XSS in the /search q Parameter

MEDIUM HIGH CONFIDENCE

OWASP A05:2025 — Injection

Location `response.end()` HTML response for `/search?q=...`: `request.url` → `new URL(...)` → `url.searchParams.get('q')` → template-literal interpolation into `<h1>Search results for ${query}</h1>` → `response.end()`

Overview The `/search` handler inserts the decoded, attacker-controlled `q` value directly into an HTML element without context-appropriate entity encoding. Because the response is served as `text/html`, HTML metacharacters in the parameter would be interpreted as markup rather than displayed as text.

Impact If exploited, an attacker could execute arbitrary same-origin JavaScript in a victim's browser, read application responses available to that origin, and perform origin-authorized actions. The current application does not use cookies or browser session storage, which limits direct session-token theft, but does not prevent script execution or access to same-origin resources.

Remediation

Encode `q` for the HTML text context before interpolation, or construct the response using a templating mechanism that escapes dynamic values by default. Add a restrictive Content-Security-Policy that disallows inline script and event-handler execution as defense in depth, while retaining output encoding as the primary fix.

Notes

- The route is public and unauthenticated, and the application does not provide a Content-Security-Policy header or equivalent meta policy. URL percent encoding is transport encoding and does not make decoded input safe for insertion into HTML.

Authentication Findings (1 finding)

AUTH-01: Missing Authentication on /api/notes/{id}

HIGH HIGH CONFIDENCE

OWASP A07:2025 — Authentication Failures

Location GET `/api/notes/{id}` in `src/server.mjs:36-47` (`createDemoServer` note route; unconditional disclosure at line 47)

Overview The note handler performs no authentication before looking up and returning a note. The optional caller-controlled `x-user-id` header is neither validated as a credential nor required, so the application never establishes a server-validated principal before disclosing a record.

Impact If exploited, an unauthenticated remote caller could retrieve complete existing note records by requesting predictable IDs, with or without supplying an arbitrary `x-user-id` value. The current data set is limited to two fixed, fictional, read-only in-memory notes, but the route's authentication design would expose any record present in that store.

Remediation

Require a server-validated authentication mechanism, such as a signed session or bearer token, before performing the note lookup. Return `401 Unauthorized` when credentials are absent or invalid, derive the user identity exclusively from the validated credential, and remove `x-user-id` as a trusted identity source.

Notes

- GET is representative, but the handler does not inspect `request.method`, so every accepted method reaches the same path-based logic. The application has no login, account, session, or token system; `alice` and `bob` are record-owner labels rather than authenticated identities.

Authorization Findings (1 finding)

AUTHZ-01: Missing Ownership Authorization on /api/notes/{id}

HIGH HIGH CONFIDENCE

OWASP A01:2025 — Broken Access Control

Location ANY (including GET, POST, PUT, PATCH, DELETE, OPTIONS, and HEAD) `/api/notes/{id}` in `src/server.mjs:36-47`

Overview The handler selects a note solely by a caller-controlled numeric path ID and verifies only that the object exists. It does not derive a trusted principal, scope the lookup by owner, or compare `note.ownerId` with the requesting user's identity before returning the record.

Impact If exploited, a caller could access another owner's complete note, including its ID, owner ID, and body, and could enumerate valid note IDs through differing success and not-found responses. The present impact is bounded by two fixed, fictional, read-only in-memory notes, but every existing record is subject to the same missing object-level authorization check.

Remediation

After authenticating the caller, enforce ownership in the data-access operation by looking up the note using both its ID and the authenticated principal's user ID, rather than retrieving by ID alone and trusting `x-user-id`. Return a consistent `404 Not Found` or policy-approved `403 Forbidden` for records outside the caller's scope, restrict the route to its intended HTTP methods, and add negative authorization tests for cross-owner and anonymous requests.

Notes

- The same handler also allows anonymous reads because `x-user-id` is optional. HEAD suppresses the response body at the Node protocol layer but still reveals record existence through the status code; other ordinary methods return the note body because the route does not enforce an HTTP method.